LÝ THUYẾT VỀ ĐỒ THỊ

Nhóm 5

Tuệ Anh
Anh Tú
Tấn Khôi

Lớp 11A3

Môn: Toán - Tin học

Khái niệm đồ thị

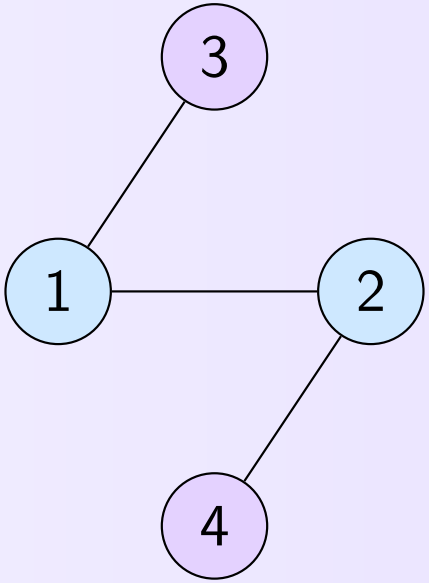
- Đồ thị là cấu trúc toán học: $G = (V, E)$
- V là tập các đỉnh (vertices)
- E là tập các cạnh (edges)
- Mỗi cạnh nối hai đỉnh biểu diễn quan hệ
- Có thể là đồ thị vô hướng hoặc có hướng

Các thành phần cơ bản

- Đỉnh (Vertex): điểm trong đồ thị
- Cạnh (Edge): kết nối giữa hai đỉnh
- Bậc đỉnh: số cạnh liên kết với đỉnh
- Đường đi (Path): dãy các đỉnh liên tiếp
- Chu trình (Cycle): đường đi khép kín

Ví dụ minh họa

- Đồ thị gồm 4 đỉnh: 1, 2, 3, 4
- Các cạnh: (1,2), (1,3), (2,4)
- Biểu diễn quan hệ giữa các đối tượng
- Có thể là mạng bạn bè
- Hoặc sơ đồ đường đi



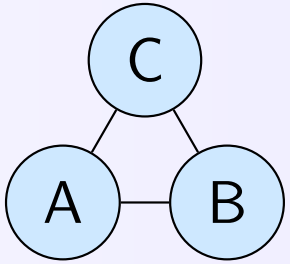
Ứng dụng của đồ thị

- Mạng xã hội (Facebook, Instagram)
- Bản đồ và hệ thống GPS
- Mạng máy tính, Internet
- Trí tuệ nhân tạo và tìm kiếm
- Bài toán đường đi ngắn nhất

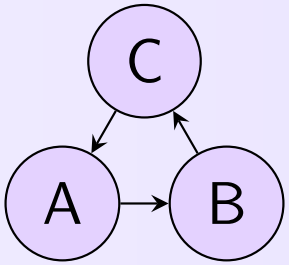
Phân loại đồ thị

- Đồ thị vô hướng (Undirected)
- Đồ thị có hướng (Directed)
- Đồ thị có trọng số (Weighted)
- Đồ thị đầy đủ (Complete)
- Đồ thị liên thông (Connected)

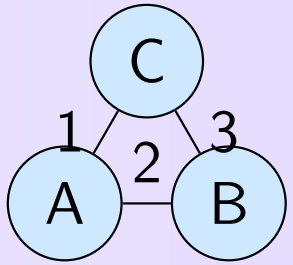
Vô hướng



Có hướng



Trọng số



Minh họa trực quan các loại đồ thị cơ bản

Ký hiệu và tính chất

- $|V|$ là số lượng đỉnh
- $|E|$ là số lượng cạnh
- $\deg(v)$ là bậc của đỉnh v
- $\sum \deg(v) = 2|E|$ (đồ thị vô hướng)
- Áp dụng trong phân tích cấu trúc đồ thị

Các loại đường đi

- Path: dãy đỉnh liên tiếp
- Simple Path: không lặp đỉnh
- Cycle: đường đi khép kín
- Euler Path: đi qua mỗi cạnh đúng 1 lần
- Hamilton Path: đi qua mỗi đỉnh đúng 1 lần

Tính chất đồ thị

- Đồ thị liên thông hay không
- Có tồn tại chu trình
- Số thành phần liên thông
- Bậc của các đỉnh
- Ảnh hưởng đến thuật toán duyệt

Mô hình hóa thực tế

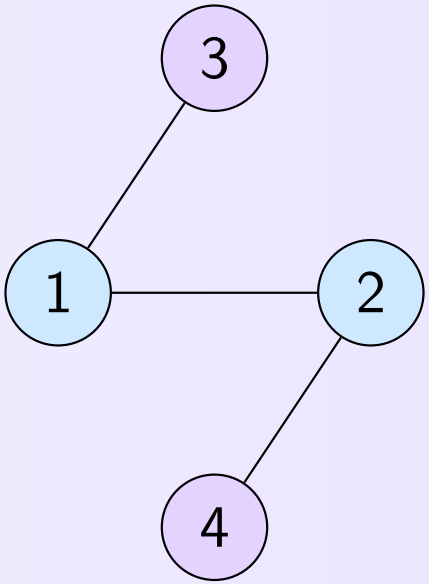
- Người dùng $\rightarrow$ đỉnh
- Quan hệ $\rightarrow$ cạnh
- Mạng xã hội (Facebook)
- Bản đồ giao thông (GPS)
- Mạng máy tính và Internet

Liên hệ thuật toán

- DFS: duyệt theo chiều sâu
- BFS: duyệt theo chiều rộng
- BFS tìm đường ngắn nhất (không trọng số)
- DFS dùng kiểm tra chu trình
- Cơ sở cho nhiều thuật toán nâng cao

Biểu diễn đồ thị

- Đồ thị $G = (V, E)$ với V là tập đỉnh, E là tập cạnh
- Đồ thị có thể vô hướng hoặc có hướng
- Biểu diễn trực quan bằng hình học (graph drawing)
- Mỗi cạnh nối hai đỉnh biểu diễn quan hệ
- Là mô hình cơ bản trong toán rời rạc



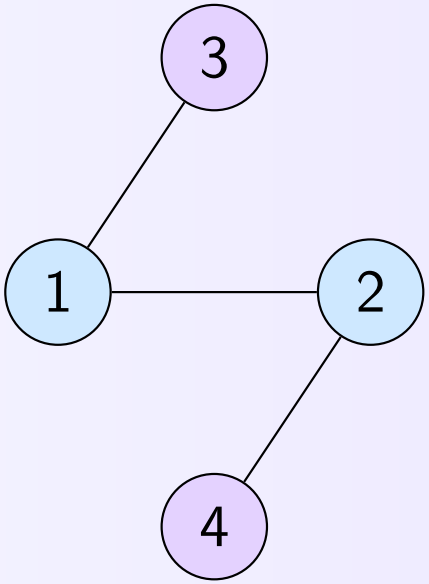
Danh sách kề (Adjacency List)

- Mỗi đỉnh lưu danh sách các đỉnh kề
- Thường cài đặt bằng vector hoặc linked list
- Tiết kiệm bộ nhớ với đồ thị thưa
- Độ phức tạp lưu trữ: $O(V + E)$
- Thuận lợi cho duyệt DFS/BFS

1: 2,3
2: 1,4
3: 1
4: 2

Minh họa danh sách kề

- Đồ thị gồm 4 đỉnh: 1, 2, 3, 4
- Các cạnh: (1,2), (1,3), (2,4)
- Biểu diễn bằng danh sách kề
- Mỗi đỉnh lưu các đỉnh kề trực tiếp
- Dễ dùng cho DFS và BFS



Danh sách kề:

- 1: 2, 3
- 2: 1, 4
- 3: 1
- 4: 2

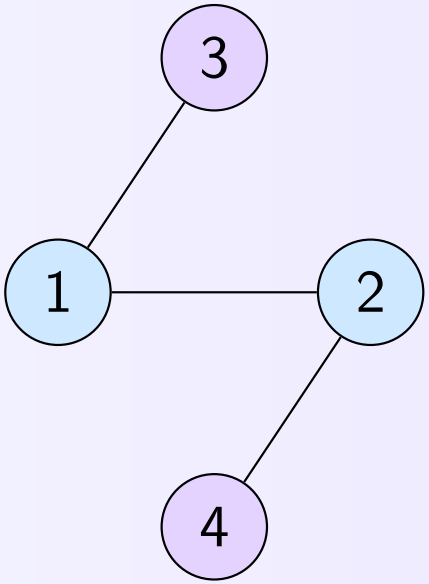
Ma trận kề (Adjacency Matrix)

- Ma trận $A[i][j]$ biểu diễn cạnh giữa i, j
- $A[i][j] = 1$ nếu có cạnh, ngược lại $= 0$
- Phù hợp với đồ thị dày
- Truy cập cạnh trong $O(1)$
- Tốn bộ nhớ $O(V^2)$

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

Minh họa ma trận kề

- Đồ thị gồm 4 đỉnh: 1, 2, 3, 4
- Các cạnh: (1,2), (1,3), (2,4)
- Biểu diễn bằng ma trận kề
- $A[i][j] = 1$ nếu có cạnh
- $A[i][j] = 0$ nếu không có cạnh



Ma trận kề:

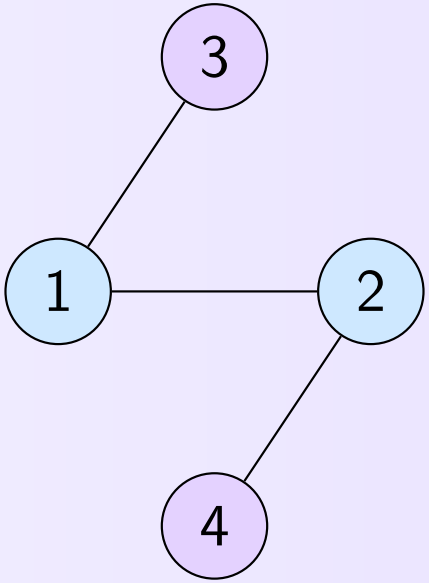
$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

Phân tích và ứng dụng DFS

- DFS là thuật toán cơ bản trong lý thuyết đồ thị
- Phù hợp với bài toán duyệt toàn bộ không gian trạng thái
- Sử dụng trong tìm kiếm và trí tuệ nhân tạo
- Là nền tảng cho nhiều thuật toán nâng cao
- Ví dụ: Tarjan, Kosaraju
- Áp dụng trong phân tích mạng
- Duyệt cây và cấu trúc dữ liệu phân cấp
- Kết hợp với BFS trong nhiều bài toán
- Tối ưu bằng đánh dấu visited
- Ý nghĩa trong Tin học và Toán học

Ví dụ minh họa DFS

- Xét đồ thị $G = (V, E)$ với $V = \{1, 2, 3, 4\}$
- Các cạnh: (1,2), (1,3), (2,4)
- Bắt đầu DFS từ đỉnh 1
- Thứ tự duyệt: $1 \to 2 \to 4 \to 3$
- Sử dụng mảng visited để tránh lặp



Ví dụ code DFS (C++)

- DFS duyệt theo chiều sâu
- Dùng đệ quy để cài đặt
- Sử dụng danh sách kề
- Có mảng visited để tránh lặp
- Độ phức tạp $O(V + E)$

```
vector<int> adj[5]; bool visited[5];
void dfs(int u)  visited[u] = true; cout « u «  ;
for(int v : adj[u])  if(!visited[v])  dfs(v);
int main()  adj[1] = 2, 3; adj[2] = 1, 4; adj[3] = 1; adj[4] = 2;
dfs(1); return 0;
```

Giải thích DFS

- Bắt đầu từ đỉnh 1
- Đi sâu: $1 \to 2 \to 4$
- Sau đó quay lui (backtrack)
- Tiếp tục duyệt đỉnh 3
- visited giúp tránh lặp
- Kết quả: 1 2 4 3

Độ phức tạp

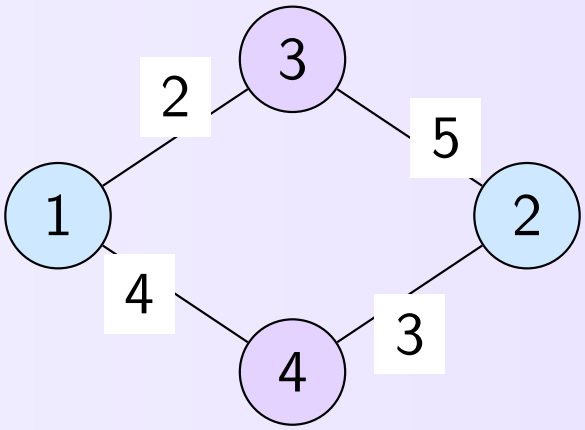
- Danh sách kề: duyệt $O(V + E)$
- Ma trận kề: duyệt $O(V^2)$
- Kiểm tra cạnh:
- List: $O(\deg(u))$
- Matrix: $O(1)$

Ứng dụng trong thuật toán

- DFS, BFS $\rightarrow$ dùng danh sách kề
- Dijkstra $\rightarrow$ list nhanh hơn
- Floyd $\rightarrow$ dùng ma trận kề
- Tối ưu phụ thuộc cấu trúc đồ thị
- Ứng dụng trong AI, mạng

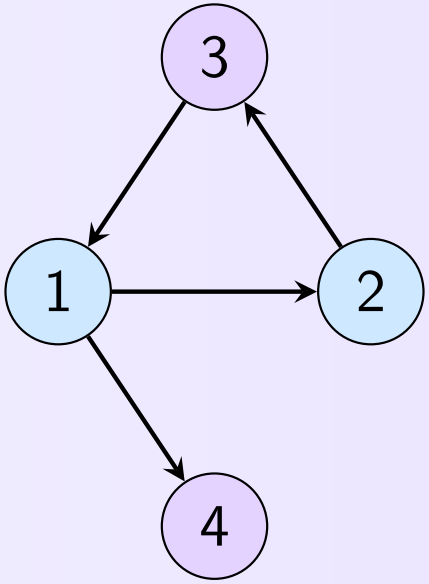
Đồ thị có trọng số

- Cạnh có giá trị trọng số
- List: lưu (đỉnh, trọng số)
- Matrix: lưu trực tiếp giá trị
- Không có cạnh $\rightarrow \infty$
- Dùng trong bài toán đường đi



Đồ thị có hướng

- Cạnh có hướng ($u \to v$)
- Ma trận: $A[i][j] \neq A[j][i]$
- List: chỉ lưu cạnh đi ra
- Áp dụng trong mạng, web
- Ví dụ: đồ thị DAG



Chuyển đổi biểu diễn

- Matrix $\rightarrow$ List: duyệt từng hàng
- List $\rightarrow$ Matrix: gán lại giá trị
- Giữ nguyên cấu trúc đồ thị
- Dùng trong tối ưu thuật toán
- Phổ biến trong lập trình thi

Phân tích bộ nhớ

- Matrix: $O(V^2)$
- List: $O(V + E)$
- Đồ thị thưa $\rightarrow$ dùng list
- Đồ thị dày $\rightarrow$ dùng matrix
- Tối ưu theo bài toán

Phép toán trên đồ thị

- Tính bậc đỉnh
- Matrix: tổng hàng/cột
- List: số phần tử kề
- Kiểm tra cạnh tồn tại
- Duyệt toàn bộ đồ thị

So sánh hai cách biểu diễn

- List: tiết kiệm bộ nhớ
- Matrix: truy cập nhanh
- List: phù hợp DFS/BFS
- Matrix: đơn giản hơn
- Chọn theo bài toán

Ý tưởng DFS

- Duyệt theo chiều sâu (depth)
- Đi càng sâu càng tốt trước
- Sử dụng đệ quy hoặc stack

Các bước thực hiện

1. Bắt đầu từ đỉnh nguồn
2. Đánh dấu đã thăm
3. Duyệt các đỉnh kề:
   - Nếu chưa thăm $\rightarrow$ gọi đệ quy

Phân tích

- Mỗi đỉnh duyệt 1 lần
- Mỗi cạnh xét tối đa 2 lần
- Thời gian: $O(n + m)$
- Bộ nhớ: $O(n)$

Code DFS C++

```
#include <bits/stdc++.h>
using namespace std;
vector<int> a[1005];
bool d[1005];
int n,m;

void dfs(int u){
    d[u]=1;
    cout<<u<<" ";
    for(int v:a[u]){
        if(!d[v]){
            dfs(v);
        }
    }
}

int main(){
```